

OOP Assignment Inspired from PakWheels

The goal of this assignment was to build a simplified version of an online car marketplace similar to PakWheels. Instead of focusing on technical complexity, I first studied how PakWheels works from a user perspective and then recreated the same main features in my program.

PakWheels essentially connects three types of people:

1. Sellers who want to list their vehicles.
2. Buyers who want to browse and purchase vehicles.
3. Admins who manage and moderate the platform.

I designed my system around these same roles so the interaction between users would resemble how PakWheels operates. In the code, each role is represented by its own class that inherits from a shared `User` base class:

```
cpp

class User {          // abstract base — shared identity for all roles
protected:
    int  userID;
    string username, email, phoneNumber;
    Address address; // every user has a location
    bool isVerified;
    ...
};

class Seller : public User { ... }; // IS-A User
class Buyer  : public User { ... }; // IS-A User
class Admin  : public User { ... }; // IS-A User
```

The `User` class holds everything common to all three roles — username, email, phone, and address. Each subclass then adds only what is specific to that role, keeping the design clean and avoiding repeated code.

Feature 1: Vehicle Listings

On PakWheels, sellers can post ads for cars or bikes with details like brand, model, year, price, and mileage.

To mimic this behavior, the system allows sellers to create vehicle listings that include similar information. These listings represent the advertisements buyers see when they browse the marketplace.

In the code, all listings share a common `Vehicle` base class that holds the core details every ad needs:

```
cpp
```

```

class Vehicle {
protected:
    int    listingID;
    string brand, model;
    int    year;
    double price;
    int    mileage;
    string registrationNumber; // no public setter — cannot be changed after posting
    bool   isApproved, isActive;

    ...
};

```

Because cars and bikes have different additional details, the system uses two separate classes that both inherit from `Vehicle`:

```

cpp

class Car : public Vehicle {
private:
    string fuelType, transmission, color, featureTag;
    int    engineCC, numDoors;

    ...
};

class Bike : public Vehicle {
private:
    string bikeType, fuelType, condition;
    int    engineCC;
    bool   hasABS;

    ...
};

```

Both `Car` and `Bike` override the `displayInfo()` function, which prints the full listing in a formatted box. The `override` keyword makes it clear these are replacing the base class version:

```

cpp

```

```

void displayInfo() const override {
    cout << " | _____ | \n";
    cout << " | CAR #" << listingID
        << (isApproved ? " [APPROVED]" : " [PENDING]") << "\n";
    cout << " | _____ | \n";
    cout << " | Brand    : " << brand    << "\n";
    cout << " | Model    : " << model    << "\n";
    cout << " | Year     : " << year     << "\n";
    cout << " | Price (PKR) : " << (long long)price << "\n";
    cout << " | Mileage   : " << mileage << " km\n";
    cout << " | Fuel      : " << fuelType  << "\n";
    cout << " | Transmission: " << transmission << "\n";
    cout << " | Engine    : " << engineCC << " cc\n";
    cout << " | Color     : " << color     << "\n";
    cout << " | Reg. No.  : " << registrationNumber << "\n";
    cout << " | _____ | \n";
}

```

Listings are added through the `Marketplace` class, which assigns each one a unique auto-incrementing ID starting from 1001:

```

cpp

int addCar(string br, string mo, int yr, double pr, int mi,
           string reg, string fuel, string trans, int eng,
           string col, int doors, string tag="Standard") {
    int id = nextID++;
    carListings[carCount++] = Car(id, br, mo, yr, pr, mi, reg,
                                   fuel, trans, eng, col, doors, tag);
    cout << " Car listing created [#" << id << "]: " << br << " " << mo << "\n";
    return id;
}

```

In `main()`, listings are created like this:

```

cpp

int cid1 = market.addCar("Toyota", "Corolla", 2021, 3200000, 45000,
                        "LHR-AA-2021", "Petrol", "Automatic", 1800, "White", 4, "Family");
int bid1 = market.addBike("Honda", "CD-70", 2022, 135000, 8000,
                        "LHR-BK-001", "Standard", 70, false, "Petrol", "Used");

```

Once a listing exists, sellers register it under their name using `addListing()`:

```

cpp

```

```
Vehicle* v1 = market.findVehicleByID(cid1);  
seller1.addListing(v1);
```

Feature 2: Admin Approval

On real marketplaces like PakWheels, listings are usually reviewed before becoming public to prevent spam or fake ads.

To simulate this, the program includes an `Admin` role that can approve or reject vehicle listings. Only approved listings appear when buyers search the marketplace.

The `Admin` class uses a default constructor that automatically creates a system administrator, since every platform needs at least one admin ready to go from the start:

```
cpp  
  
Admin() : User(1, "sysadmin", "admin@pw.pk", "0300-0000000",  
             Address("Karachi", "Sindh", "Pakistan", "I.I. Chundrigar", 75600)),  
         adminLevel("SuperAdmin"), listingsApproved(0),  
         listingsRejected(0), usersVerified(0), canDeleteUsers(true)  
{ isVerified = true; }
```

Approving a listing sets its `isApproved` flag to `true`. The function first checks if it has already been approved to avoid double-counting:

```
cpp  
  
void approveListing(Vehicle* v) {  
    if (!v) return;  
    if (v->getApproved()) { cout << " Already approved.\n"; return; }  
    v->approve();  
    listingsApproved++;  
    cout << " APPROVED Listing #" << v->getID() << " by " << username << "\n";  
}
```

Rejecting a listing calls `deactivate()`, which sets `isActive` to `false`. The listing stays in memory but is invisible to buyers:

```
cpp
```

```

void rejectListing(Vehicle* v) {
    if (!v) return;
    v->deactivate();
    listingsRejected++;
    cout << " REJECTED Listing #" << v->getID() << " by " << username << "\n";
}

```

In `main()`, the admin reviews all new listings:

```

cpp

admin.approveListing(market.findVehicleByID(cid1)); // Toyota Corolla — approved
admin.approveListing(market.findVehicleByID(cid2)); // Honda Civic — approved
admin.rejectListing(market.findVehicleByID(cid4)); // Hyundai Tucson — rejected

```

The admin also keeps track of everything through an audit counter and a dashboard function:

```

cpp

void showStats() const {
    cout << " Level   : " << adminLevel << "\n";
    cout << " Approved : " << listingsApproved << "\n";
    cout << " Rejected  : " << listingsRejected << "\n";
    cout << " Verified  : " << usersVerified << " users\n";
}

```

Feature 3: Vehicle Search

A key feature of PakWheels is that buyers can search for vehicles based on criteria like brand, price range, or model year.

The program replicates this behavior by allowing buyers to search vehicles using filters such as brand, price range, and year.

The filter logic lives inside the `Vehicle` class as a constant function called `matchesFilter()`. It checks the vehicle's attributes against whatever filter values are provided, and returns `false` the moment any condition fails:

```

cpp

```

```
bool matchesFilter(string fb, string fm, double lo, double hi,
                  int ylo, int yhi, int mxMi) const {
    if (!fb.empty() && brand != fb) return false; // brand filter
    if (!fm.empty() && model != fm) return false; // model filter
    if (price < lo || price > hi) return false; // price range
    if (year < ylo || year > yhi) return false; // year range
    if (mileage > mxMi) return false; // mileage cap
    return true;
}
```

The `Marketplace::searchVehicles()` function loops through both the car and bike arrays, and only shows listings that are both active and approved:

```
cpp

void searchVehicles(string fb="", string fm="",
                   double lo=0, double hi=1e10,
                   int ylo=1980, int yhi=2030, int mxMi=999999) const {
    for (int i = 0; i < carCount; i++) {
        if (carListings[i].getActive() && carListings[i].getApproved() &&
            carListings[i].matchesFilter(fb, fm, lo, hi, ylo, yhi, mxMi)) {
            cout << " [CAR] " << carListings[i].getSummary() << "\n";
        }
    }
    for (int i = 0; i < bikeCount; i++) {
        if (bikeListings[i].getActive() && bikeListings[i].getApproved() &&
            bikeListings[i].matchesFilter(fb, fm, lo, hi, ylo, yhi, mxMi)) {
            cout << " [BIKE] " << bikeListings[i].getSummary() << "\n";
        }
    }
}
```

All parameters have default values, so a search with no arguments shows everything, and partial filters work naturally. In `main()`, three different searches are demonstrated:

```
cpp

// Filter by brand only
market.searchVehicles("Toyota");

// Filter by price range only
market.searchVehicles("", "", 1000000, 4000000);

// Filter by year and mileage together
market.searchVehicles("", "", 0, 1e10, 2022, 2030, 25000);
```

Feature 4: Favorites

PakWheels allows users to save ads they are interested in.

To represent this behavior, buyers in the system can save vehicles as favorites, allowing them to easily access the listings they like.

The `Buyer` class holds an array of `Vehicle` pointers for this purpose. These are pointers — not copies — because the buyer is simply bookmarking a listing that already exists in the marketplace. This is an example of **aggregation**:

```
cpp

class Buyer : public User {
private:
    Vehicle* favorites[MAX_FAVORITES]; // stores references, not owned copies
    int    favCount;
    ...
};
```

`saveFavorite()` checks for duplicates before adding, so the same listing cannot be saved twice:

```
cpp

bool saveFavorite(Vehicle* v) {
    if (favCount >= MAX_FAVORITES) {
        cout << " Favorites full.\n"; return false;
    }
    for (int i = 0; i < favCount; i++)
        if (favorites[i]->getID() == v->getID()) {
            cout << " Already saved.\n"; return false;
        }
    favorites[favCount++] = v;
    cout << " Saved: " << v->getSummary() << "\n";
    return true;
}
```

`removeFavorite()` finds the listing by ID and shifts the remaining elements to close the gap:

```
cpp
```

```

bool removeFavorite(int lid) {
    for (int i = 0; i < favCount; i++) {
        if (favorites[i]->getID() == lid) {
            for (int j = i; j < favCount - 1; j++)
                favorites[j] = favorites[j + 1];
            favorites[--favCount] = nullptr;
            cout << "  Removed favorite #" << lid << "\n";
            return true;
        }
    }
    cout << "  Not found in favorites.\n"; return false;
}

```

In `main()`, buyers save and remove favorites like this:

```

cpp

buyer1.saveFavorite(v1); // save Toyota Corolla
buyer1.saveFavorite(v2); // save Honda Civic
buyer1.showFavorites();  // display saved list

buyer1.removeFavorite(cid2); // remove Honda Civic
buyer1.showFavorites();      // updated list

```

Feature 5: Messaging Between Users

Another common feature in marketplaces is the ability for users to contact sellers or other users regarding a listing.

To simulate this interaction, the program includes a simple messaging system where buyers can send messages related to a specific vehicle listing.

Each message is represented by the `Message` class, which stores who sent it, who it is for, what it says, and which listing it is about:

```

cpp

class Message {
private:
    int    messageID, listingID;
    string senderName, receiverName, content;
    string timestamp; // no setter — cannot be changed after the message is sent
    bool   isRead;
    ...
};

```


`timestamp` has a getter but deliberately no setter. The time a message was sent is a fixed historical fact and should never be editable.

The `Marketplace::sendMessage()` function creates the message, stores a copy in the platform log, and delivers it directly to the recipient's inbox:

```
cpp

bool sendMessage(Buyer* from, Buyer* to, string content, int lid) {
    Message m(nextMsgID++, from->getUsername(),
               to->getUsername(), content, lid, "2024-06-01 14:30");
    msgLog[msgLogCount++] = m;
    to->receiveMessage(m);
    cout << " Message sent: " << from->getUsername()
          << " -> " << to->getUsername() << "\n";
    return true;
}
```

Each `Buyer` owns a `Message` inbox array directly inside the object. This is **composition** — the messages belong to the buyer and are destroyed with them:

```
cpp

class Buyer : public User {
private:
    Message inbox[MAX_MESSAGES]; // owned by this buyer
    int msgCount;
    ...

    bool receiveMessage(const Message& m) {
        if (msgCount >= MAX_MESSAGES) { cout << " Inbox full.\n"; return false; }
        inbox[msgCount++] = m;
        return true;
    }

    void showInbox() const {
        for (int i = 0; i < msgCount; i++) inbox[i].display();
    }
};
```

In `main()`, a message is sent and the recipient's inbox is displayed:

```
cpp

market.sendMessage(&buyer1, &buyer2,
                  "Is the CD-70 available? Can we negotiate?", bid1);
buyer2.showInbox();
```

Feature 6: Listing Management

On PakWheels, sellers can manage their ads, including removing or updating them.

Similarly, sellers in the system can:

- View their listings
- Remove listings
- Track sales revenue

The `Seller` class holds an array of `Vehicle` pointers representing the listings it has posted. These are pointers to vehicles that also exist in the `Marketplace` — both the seller and the marketplace reference the same object.

This is **aggregation**:

```
cpp

class Seller : public User {
private:
    Vehicle* myListings[MAX_SELLER_ADS]; // shared references with Marketplace
    int    listingCount;
    double totalRevenue;
    string  businessName;
    bool    isPremium;

    ...
};
```

`removeListing()` finds the listing by ID, deactivates the vehicle so it disappears from search results, then removes the pointer from the seller's array:

```
cpp

bool removeListing(int lid) {
    for (int i = 0; i < listingCount; i++) {
        if (myListings[i]->getID() == lid) {
            myListings[i]->deactivate();
            for (int j = i; j < listingCount - 1; j++)
                myListings[j] = myListings[j + 1];
            myListings[--listingCount] = nullptr;
            cout << " Listing #" << lid << " removed.\n";
            return true;
        }
    }
    cout << " Listing not found.\n"; return false;
}
```

To update a listing's price, the `Marketplace` exposes an `updateCarPrice()` function that uses the `setPrice()` setter on the vehicle:

```
cpp

bool updateCarPrice(int id, double np) {
    Car* c = findCarByID(id);
    if (!c) { cout << " Car #" << id << " not found.\n"; return false; }
    double old = c->getPrice();
    c->setPrice(np);
    cout << " Price updated #" << id << ": "
         << (long long)old << " -> " << (long long)np << "\n";
    return true;
}
```

To delete a listing entirely, `deleteListing()` deactivates the vehicle regardless of whether it is a car or a bike, using a base class `Vehicle*` pointer:

```
cpp

bool deleteListing(int id) {
    Vehicle* v = findVehicleByID(id);
    if (!v) { cout << " Not found.\n"; return false; }
    v->deactivate();
    cout << " Listing #" << id << " deactivated.\n";
    return true;
}
```

Sellers can also log completed sales and track cumulative revenue:

```
cpp

void recordSale(double amt) {
    totalRevenue += amt;
    cout << " Sale PKR " << (long long)amt
         << " recorded. Total: PKR " << (long long)totalRevenue << "\n";
}
```

In `main()`, all three listing management operations are used:

```
cpp

market.updateCarPrice(cid3, 1750000); // reduce Swift price
market.deleteListing(cid2);           // remove Civic from platform
seller1.removeListing(cid3);          // seller removes their own Swift
seller1.showMyListings();             // view remaining listings
seller2.recordSale(5500000);          // log a completed sale
```

Overall Design Idea

Instead of replicating every feature of PakWheels, the goal was to capture the core workflow of the platform:

1. Sellers create listings
2. Admin reviews listings
3. Buyers search vehicles
4. Buyers save favorites
5. Users communicate through messages

This simplified design models the basic ecosystem of a real online vehicle marketplace, while keeping the system manageable for an Object-Oriented Programming assignment. Each feature maps directly to one or more classes, and the relationships between those classes — who owns what, who references what — reflect the real-world relationships between the roles on the actual platform.